EY Techathon 5.0

Retail [ABFRL] - AI Sales Agent

Judge Evaluation Analysis & Preparation Report

Comprehensive project assessment against problem statement, anticipated jury questions, and readiness evaluation

Project: RetailBuddy - AI-Driven Conversational Sales Agent
Technology Stack: LangGraph + GPT-4o-mini + FastAPI + React + Twilio
Channels: Web Chat | WhatsApp | In-Store Kiosk

1. PROJECT STRENGTHS

Evaluation Summary Table

Evaluation Parameter	Your Coverage	Verdict
Scope of Problem Covered	Covers ~85% of the problem statement	Strong
Solution Design & Tech	LangGraph ReAct agent + 12 tool agents + WhatsApp + Kiosk + Admin	Very Strong
Uniqueness & Innovation	Real-time WS streaming timeline, autonomous upsell, tier-based ranking	Strong
End-user Benefit & Impact	Multi-persona dashboards (Customer, StoreKeeper, Admin), omnichannel	Good

Key Differentiators (What Sets You Apart)

-> 12 specialized @tool agents orchestrated by GPT-4o-mini via LangGraph ReAct loop - this is genuinely agentic AI, not just prompt chaining.

-> Real-time Agent Timeline streamed via WebSocket - judges can visually see the orchestration happening live with timeline_start, timeline_log, timeline_end events.

-> 3 channels working: Web Chat (WebSocket), WhatsApp (Twilio), In-Store Kiosk - true omnichannel.

-> Omnichannel cart transfer: switch_to_online tool lets kiosk items move to online checkout and vice versa.

-> Autonomous upsell_agent - auto-triggers cross-sell recommendations after viewing cart without user asking.

-> Customer tier-based product ranking: tier_1 (premium) customers see expensive items first, tier_3 (budget) customers see affordable options first.

-> Admin AI Insights: GPT-4o-mini analyzes actual store metrics and generates strategic business recommendations.

-> Google Maps integration for nearest store locator with pre-computed distance table.

2. SOLUTION ARCHITECTURE

Agent Tools (12 Specialized Worker Agents)

#	Tool Name	Purpose
1	customer_info_agent	Fetches customer profile (name, phone, address, loyalty) or order history
2	inventory_agent	Primary product search with keyword scoring, gender/price filtering, tier-based sorting, dedup
3	recommendation_agent	Display layer - formats raw inventory results into UI-friendly product cards with images
4	add_to_cart_agent	Adds product to cart table with stock validation, discount calc, unique order ID generation
5	view_cart_agent	Lists all pending cart items with prices, discounts, savings, and total amount
6	remove_from_cart_agent	Deletes a specific cart item by order_id, verifying customer ownership
7	choose_channel_agent	Presents online vs offline purchase channel choice with cart summary
8	payment_agent	Two-stage: (1) show 4 payment options, (2) process payment, update cart status to paid
9	fulfillment_agent	Online: confirms home delivery. Offline: assigns store, sends WhatsApp pickup confirmation
10	loyalty_agent	Awards loyalty points based on product loyalty_points x quantity, generates coupon codes
11	post_purchase_agent	DB finalizer: creates OrderHistory + StoreKeeperTransaction records, deducts inventory
12	upsell_agent	Autonomous cross-sell: maps cart categories to complementary items (Shirt->Belt/Shoes)

LangGraph Architecture

Main Agent Graph (intelligent_agent_graph.py): Uses a ReAct (Reasoning + Acting) loop pattern with 2 nodes:
START -> [agent] -> (should_continue?) -> [tools] -> [agent] -> ... -> END

The 'agent' node invokes GPT-4o-mini with all 12 tools bound. A ~3000-word system prompt encodes the entire shopping workflow. The 'tools' node executes requested tools and returns results. Routing: if last AI message has tool_calls -> route to tools, else -> END.

Store Kiosk Graph (store_kiosk_graph.py): Deterministic linear pipeline (no LLM decision-making):
START -> welcome -> payment_prompt -> payment_method -> fulfillment -> loyalty -> post_purchase -> END

WhatsApp Graph (intelligent_whatsapp_graph.py): Full clone of main agent graph with Twilio I/O layer. Responses formatted as WhatsApp text cards + product images sent as separate media messages.

3. GAPS & WEAKNESSES

Gap Analysis vs Problem Statement

Gap	Requirement	Your Status
Voice Assistant	'voice assistant' channel mentioned in PS	MISSING entirely
Session Continuity	'customer switches from mobile to kiosk'	PARTIAL - cart transfers, but chat history doesn't transfer across channels
Payment Gateway Stub	'Dummy API for authorizations, captures, declined transactions'	BASIC - payment is just status update, no simulated failure/retry
10+ Customer Profiles	>=10 customers with demographics, history, loyalty tier	Need to verify data completeness
Reserve-in-Store	'reserve in-store for try-on'	PARTIAL - offline booking exists but no explicit try-on reservation
Shipment Tracking	Post-Purchase Agent should 'track shipments'	MISSING - no tracking simulation
Returns/Exchanges	'Handles returns/exchanges'	COVERED - Return initiation with 24h cooling + monitoring agent
POS Integration	'barcode scan and payment'	COVERED - Barcode panel in kiosk

4. ANTICIPATED JURY QUESTIONS

RED CIRCLE CRITICAL Questions (Very Likely to Be Asked)

Q1: How does session continuity work when a customer moves from web chat to the in-store kiosk?

Suggested Answer: Cart items persist in the DB with a 'channel' field. When the customer logs into the kiosk, welcome_node fetches all their pending offline cart items. The switch_to_online tool reverses the direction. However, conversation context (chat history) does NOT transfer between channels - be honest about this limitation. In production, we'd add a session service that shares summarized context.

Q2: Walk us through what happens when a payment fails. How does the agent recover?

Suggested Answer: We validate inputs and handle edge cases, but a full retry flow with exponential backoff was deprioritized in favor of other features. Our architecture supports adding it as another @tool function. The payment_agent currently handles method selection and processing, but doesn't simulate declined transactions.

Q3: How are the Worker Agents actually 'loosely coupled'? Can you add a new agent easily?

Suggested Answer: Each agent is a @tool decorated Python function. Adding a new agent (e.g., Gift Wrapping Agent) requires: (1) write one new function, (2) decorate with @tool, (3) add to the tools list. The LLM automatically decides when to invoke it based on the system prompt. No graph rewiring needed. This is the power of the ReAct pattern.

Q4: What LLM are you using, and why? What's the latency like?

Suggested Answer: GPT-4o-mini, temperature 0.1. Chosen for cost-efficiency (10x cheaper than GPT-4o) and speed (~1-2s per tool call). WebSocket streaming with chunked delta delivery (24 chars/chunk) reduces perceived latency. The streaming agent timeline gives visual feedback during processing.

Q5: How does the Recommendation Agent personalize recommendations?

Suggested Answer: Three personalization layers: (1) Customer tier ranking - premium customers see premium products first, budget customers see affordable options. (2) Gender-aware filtering - inferred from customer profile or explicit query. (3) Purchase history exclusion - already-bought products filtered out. (4) Relevance scoring across product name, category, season, festival.

YELLOW CIRCLE PROBABLE Questions (Likely to Be Asked)

Q6: How do you prevent hallucinations? The LLM could recommend non-existent products.

Suggested Answer: The LLM never generates product data. It only decides WHICH tool to call. The inventory_agent tool queries the SQLite database directly - all product data comes from our database, not from the LLM's knowledge. This is a key architectural decision: the LLM is an orchestrator, not a data source.

Q7: What's the upsell/cross-sell strategy? How much AOV lift would you expect?

Suggested Answer: The upsell_agent automatically triggers after cart view. It maps categories to complementary items (Shirt->Belt/Shoes/Watches for men, Kurti->Footwear/Jewellery for women). Combined with tier-based pricing and loyalty discounts, this drives AOV. The agent uses gender-aware complementary category mappings.

Q8: How does the Admin dashboard help business stakeholders?

Suggested Answer: Real-time analytics with: online/offline transaction split, store-level performance, monthly trends, per-customer analysis, AND AI-powered strategic insights generated by GPT-4o-mini analyzing the actual store metrics. Store keepers also get their own per-store dashboard with AI insights and benchmarking.

Q9: Can you demonstrate the WhatsApp flow live?

Suggested Answer: If Twilio is configured with ngrok tunnel, demo it live. If not: 'We have the full integration built with Twilio - message parsing, product card formatting with emojis, image delivery. The WhatsApp agent reuses the same LangGraph as web chat. Due to demo environment constraints, I'll show the code + architecture.'

Q10: What about data privacy? You're sending customer data to OpenAI.

Suggested Answer: In production, we'd use Azure OpenAI with data residency guarantees. The current prototype uses OpenAI API for rapid development. Customer PII in prompts can be masked. The system prompt instructs the LLM on behavior - actual customer data flows through tool calls which are local DB queries.

GREEN CIRCLE BONUS Questions (If They Dig Deep)

Q11: Why LangGraph instead of LangChain agents or AutoGen?

Suggested Answer: LangGraph gives us explicit state management and deterministic tool execution order. Unlike vanilla LangChain agents, we can control the conversation flow, persist state across turns, and build hybrid architectures (ReAct for web, linear for kiosk). The StateGraph with typed state (AgentState) ensures type safety and predictable behavior.

Q12: What happens if two kiosk users try to buy the same last-in-stock item?

Suggested Answer: We check stock at add-to-cart time via product.quantity validation. However, there's no database-level pessimistic locking - a race condition is theoretically possible. For production: we'd add row-level locking or a reservation system with TTL. SQLite's write serialization partially mitigates this in the prototype.

Q13: How scalable is this? You're using SQLite and in-memory state.

Suggested Answer: SQLite is for prototyping. The SQLModel/SQLAlchemy ORM makes swapping to PostgreSQL a config change. In-memory session state (_WHATSAPP_SESSIONS, _SHOWN_PRODUCT_IDS) would move to Redis. FastAPI supports horizontal scaling with Uvicorn workers. The stateless REST endpoints are already horizontally scalable.

5. API ENDPOINTS OVERVIEW

Key REST Endpoints

Method	Path	Purpose
POST	/api/chat	Primary chat endpoint (HTTP, non-streaming)
WS	ws://localhost:8000/ws/chat	Real-time agentic chat with streaming timeline
POST	/api/kiosk/chat	Kiosk chat endpoint (linear flow)
POST	/webhook	Twilio WhatsApp webhook
GET	/api/admin/dashboard	Comprehensive admin analytics
GET	/api/admin/ai-insights	LLM-powered business insights
GET	/api/store/{id}/transactions	Store-specific transaction history
GET	/api/store/{id}/ai-insights	LLM-powered per-store insights
POST	/api/orders/initiate-return	Return with 24h cooling period
POST	/api/orders/return-monitoring	Batch agent - auto-completes returns
GET	/api/locate-us/{cust}	Nearby stores with distance/map URL
POST	/api/cart/apply-loyalty	Apply loyalty points (1 pt = Rs.1)
POST	/api/wishlist/toggle	Toggle product in wishlist
POST	/api/cart/switch-to-online	Switch offline cart to online
GET	/api/customer/{id}/{role}	Customer/storekeeper profile by role
POST	/api/feedback/{pid}/{cid}	Submit 1-5 star rating

6. FINAL VERDICT & READINESS

Score Estimate (Per Evaluation Parameter)

Parameter	Score	Notes
Scope of problem covered	8 / 10	Missing voice channel, partial session continuity across channels
Solution design & technology	9 / 10	Excellent LangGraph architecture, clean separation, real DB, 12 tools
Uniqueness & innovation	8 / 10	Streaming timeline, autonomous upsell, tier-based ranking are differentiators
End-user benefit & impact	7.5 / 10	Multi-persona coverage great, demo polish matters for final impression

OVERALL: 8.1 / 10 - STRONG FINALIST CANDIDATE

Top 3 Action Items Before Semi-Final

1. POLISH THE DEMO FLOW: Practice a 5-minute scripted demo: Customer searches -> product cards appear -> add to cart -> upsell triggers -> payment -> kiosk pickup -> show WhatsApp message -> Admin dashboard. The streaming timeline is your WOW moment - make sure it works smoothly.

2. PREPARE FOR SESSION CONTINUITY QUESTION: This is your biggest gap vs the problem statement. Have a clear answer: 'Cart data persists across channels. Conversation context is channel-specific by design - in production, we'd add a session service that shares summarized context.'

3. HAVE EDGE-CASE DEMOS READY: Show an out-of-stock scenario (agent handles it gracefully), show a return initiation (24h cooling period), and show the upsell auto-triggering after viewing cart. These demonstrate robustness and thoughtful engineering.

Presentation Sequence (10 minutes)

Minute 0-1: Quick intro - team, problem summary, solution overview (1 slide)
Minute 1-2: Architecture slide - show the multi-agent diagram with 12 tools + 3 channels
Minute 2-7: LIVE DEMO (this is where you win or lose)
- Customer web chat: search -> recommend -> add to cart -> upsell triggers
- Show Agent Timeline streaming in real-time
- Cart -> Payment -> Fulfillment -> Loyalty -> Post-purchase
- Kiosk: customer logs in, sees offline cart, pays
- Admin dashboard with AI insights
- (Optional) WhatsApp if Twilio is live

Minute 7-9: Value proposition slide - AOV impact, conversion rate improvement
Minute 9-10: Summary + future roadmap (voice, session sync, production scaling)

KEY TAKEAWAY: You have a genuinely strong project. The multi-agent architecture

with LangGraph, real database, 3 channels, and streaming timeline puts you well above average. Focus your presentation on the LIVE DEMO - that's where you'll win.